GUÍA DE ESTUDIO · NIVEL PRINCIPIANTE

Tx_Series

De datos.php a una base de datos MySQL

Orden de trabajo paso a paso y explicación de cada parte del proyecto

Esta guía explica el proyecto **tal y como está ahora**: un listado de series que lee y guarda los datos en MySQL. Sirve para entenderlo, montarlo desde cero y probarlo sin liarte.

Proyecto: Tx_Series · Listado, búsqueda, estadísticas y alta de series
Tecnologías: PHP · MySQL · HTML · CSS · Laragon
Fecha: Noviembre de 2026

Índice

1. Qué hace el proyecto y qué ha cambiado	3
2. Entorno y puesta en marcha	4
3. Mapa de archivos	4
4. Ruta de trabajo en orden	5
5. Paso 1 — Crear la base de datos	6
6. Paso 2 — Conectar PHP con MySQL	8
7. Paso 3 — Leer y guardar series (funciones)	9
8. Paso 4 — Página de inicio	11
9. Paso 5 — Página de búsqueda	12
10. Paso 6 — Página de estadísticas	14
11. Paso 7 — Página de añadir serie	15
12. Recorrido completo de los datos	17
13. Pruebas de funcionamiento	18
14. Problemas frecuentes y solución	19
15. Diccionario de nombres en euskera	19
16. Checklist final y cómo explicar el ejercicio	21
Anexo — Comandos SQL útiles	22

Cómo usar esta guía

> **Lee en orden y no saltes pasos.** Cada paso explica qué se hace, por qué se hace y cómo comprobar que está bien. Si algo falla, para y revisa ese paso antes de seguir.

- Los bloques con fondo gris son **código real del proyecto**.
- Los recuadros azules son **explicaciones**.
- Los recuadros naranjas son **avisos importantes**.
- Los recuadros verdes son **comprobaciones** de que todo va bien.
- Las etiquetas ANTES y AHORA indican qué había antes y qué hay ahora.

1. Qué hace el proyecto y qué ha cambiado

Tx_Series es una web sencilla para guardar series de televisión. Tiene cuatro páginas:

Página	Para qué sirve
`index.php`	Muestra todas las series guardadas.
`search.php`	Busca series por su título.
`add_series.php`	Añade una serie nueva.
`statistics.php`	Muestra número de series, series en marcha, media y mejor valorada.

Qué había antes y qué hay ahora

Parte	Antes	Ahora
Dónde están los datos	En un array dentro de `includes/datuak.php`.	En la tabla `serieak` de la base de datos `tx_series`.
Cómo se cargan	Con `require "includes/datuak.php"`.	Con `$series = serieakLortu($conn);`.
Dónde se guarda una serie nueva	En la sesión (`$_SESSION["series"]`).	Con un `INSERT` en MySQL.
Qué pasa al cerrar el navegador	Los datos podían perderse al caducar la sesión.	Los datos siguen guardados en la base de datos.
Funciones de mostrar, buscar y calcular	Recibían un array.	Igual: siguen recibiendo un array. No se han tocado.

> **Idea clave:** solo ha cambiado *de dónde salen* y *dónde se guardan* las series. Las funciones que muestran, buscan y calculan siguen funcionando igual porque reciben un array.

Los tres archivos nuevos

Archivo	Qué es	¿Se usa al abrir una página?
`includes/datuak.sql`	Script que crea la base de datos, la tabla y las 8 series iniciales.	No. Se ejecuta una sola vez para preparar MySQL.
`includes/konexioa.php`	Se conecta a MySQL y guarda la conexión en `$conn`.	Sí, en todas las páginas.
`includes/datuak.php`	El archivo antiguo con el array. Ya no se carga.	No. Solo queda como referencia.

> **No confundas `datuak.php` con `datuak.sql`.** `datuak.php` ya no se usa. `datuak.sql` no lo usan las páginas, pero es necesario para crear la base de datos desde cero.

2. Entorno y puesta en marcha

El proyecto funciona con **Laragon**, que trae Apache, PHP y MySQL juntos. Antes de tocar nada, comprueba que todo está encendido.

Arrancar Laragon

1. Abre Laragon.
2. Pulsa el botón **Start All**.
3. Espera a que Apache y MySQL estén en verde.

Direcciones que vas a usar

Qué	Dirección
La web	`http://localhost/Control/Control/index.php`
phpMyAdmin	`http://localhost/phpmyadmin`
Carpeta del proyecto	`C:\laragon\www\Control\Control`

Datos de conexión

Dato	Valor	Qué significa
Servidor	`localhost`	MySQL está en tu propio ordenador.
Usuario	`root`	Usuario administrador por defecto de Laragon.
Contraseña	(vacía)	Laragon no pone contraseña por defecto.
Base de datos	`tx_series`	Nombre de la base de datos del proyecto.

> **Esto es solo para practicar en tu ordenador.** Usar `root` sin contraseña está bien en Laragon, pero nunca se hace así en una web publicada en Internet.

> **Comprobación:** abre phpMyAdmin y mira que aparece `tx_series` en la lista de la izquierda. Si aparece, MySQL está funcionando y la base de datos ya está creada.

Nombre de las carpetas

Fíjate en que el proyecto está dentro de `www\Control\Control`. Por eso la dirección lleva `/Control/Control/` dos veces. No es un error: son dos carpetas con el mismo nombre.

3. Mapa de archivos

Esta es la estructura actual del proyecto:

```
Control/
├── index.php          → página de inicio (lista de series)
├── search.php         → página de búsqueda
├── add_series.php     → formulario para añadir una serie
├── statistics.php     → página de estadísticas
├── header.php         → cabecera y menú de navegación
├── footer.php         → pie de página
├── css/
│   └── styles.css     → estilos
├── img/               → imágenes de las series y el logo
└── includes/
    ├── konexioa.php   → NUEVO: conexión a MySQL
    ├── funtzioak.php  → funciones del proyecto
    ├── datuak.sql     → NUEVO: crea la base de datos y los datos
    └── datuak.php     → ANTIGUO: ya no se usa
```

Qué hace cada archivo

Archivo	Función	¿Se usa ahora?
`index.php`	Pide las series a la base de datos y las muestra.	Sí
`search.php`	Pide las series y filtra por título.	Sí
`add_series.php`	Muestra el formulario y guarda la serie en MySQL.	Sí
`statistics.php`	Pide las series y calcula las estadísticas.	Sí
`header.php`	Parte HTML de arriba: título, logo y menú.	Sí
`footer.php`	Parte HTML de abajo.	Sí
`includes/konexioa.php`	Crea `$conn`, la conexión con MySQL.	Sí, siempre
`includes/funtzioak.php`	Funciones para leer, insertar, mostrar, buscar y calcular.	Sí
`includes/datuak.sql`	Script de creación de la base de datos.	Solo para instalarla
`includes/datuak.php`	Array con los datos originales.	**No**

> **Regla fácil:** los archivos de la carpeta principal son páginas que ves en el navegador. Los de `includes/` son piezas que se cargan desde esas páginas y no se abren directamente.

4. Ruta de trabajo en orden

Si tuvieras que hacer el ejercicio desde cero, este es el orden correcto. La regla de oro es: **termina y prueba un paso antes de empezar el siguiente**.

Orden	Qué haces	Cómo sabes que está bien
1	Leer el proyecto antiguo y entender qué datos hay.	Sabes qué significa cada campo.
2	Arrancar Laragon y abrir phpMyAdmin.	Apache y MySQL en verde.
3	Diseñar la tabla: campos y tipos.	Tienes claros los 7 campos.
4	Escribir `datuak.sql`.	El archivo tiene CREATE, USE, CREATE TABLE e INSERT.

5	Ejecutar el SQL en phpMyAdmin.	Aparece la base de datos `tx_series`.
6	Comprobar la tabla y los datos.	La tabla `serieak` tiene 8 filas.
7	Crear `konexioa.php`.	La web no muestra error de conexión.
8	Adaptar `funtzioak.php`.	Existen `serieakLortu`, `serieaLortu` y `serieaGehitu`.
9	Adaptar `index.php`.	Se ven las 8 series.
10	Adaptar `search.php`.	Buscar «Dark» devuelve una serie.
11	Adaptar `statistics.php`.	Los números coinciden.
12	Adaptar `add_series.php`.	La serie nueva aparece y sigue ahí al recargar.
13	Probar todo el recorrido.	Las cuatro páginas funcionan.
14	Revisar comentarios y limpiar.	No queda ninguna carga de `datuak.php`.

Por qué este orden

- **Primero la base de datos**, porque sin tabla no hay nada que leer.
- **Después la conexión**, porque sin conexión las consultas fallan.
- **Después las funciones**, porque las páginas las utilizan.
- **Al final las páginas**, de la más sencilla (inicio) a la más completa (añadir).

> **Error típico de principiante:** cambiar todas las páginas a la vez y luego no saber qué falla. Hazlo de una en una y recarga el navegador después de cada cambio.

5. Paso 1 — Crear la base de datos

Una base de datos es como un armario con cajones. La base se llama `tx_series` y dentro hay una tabla llamada `serieak`. Cada fila de la tabla es una serie.

Los campos de la tabla

Campo	Tipo	Qué guarda	Ejemplo
`id`	INT, autoincremental	Número único de cada serie.	1, 2, 3...
`izenburua`	VARCHAR(100)	Título de la serie.	Dark
`generoa`	VARCHAR(50)	Género.	Drama
`denboraldiak`	INT	Número de temporadas.	3
`balorazioa`	DECIMAL(3,1)	Nota con un decimal.	8.7
`amaituta`	TINYINT(1)	0 = en marcha, 1 = terminada.	1
`irudia`	VARCHAR(255)	Ruta de la imagen.	img/dark.jpg

El archivo `includes/datuak.sql`

Este es el contenido completo del archivo:

```sql
-- Creamos la base de datos (si ya existe, no da error).
CREATE DATABASE IF NOT EXISTS tx_series;

-- Le decimos a MySQL que trabaje con esta base de datos.
USE tx_series;

-- Creamos la tabla de series (si ya existe, no da error).
CREATE TABLE IF NOT EXISTS serieak (
    id INT AUTO_INCREMENT PRIMARY KEY,
    izenburua VARCHAR(100) NOT NULL,
    generoa VARCHAR(50) NOT NULL,
    denboraldiak INT NOT NULL,
    balorazioa DECIMAL(3,1) NOT NULL,
    amaituta TINYINT(1) NOT NULL,
    irudia VARCHAR(255)
);

-- Metemos las mismas series que habia en datos.php.
-- amaituta: 0 = martxan, 1 = amaituta.
INSERT INTO serieak (izenburua, generoa, denboraldiak, balorazioa, amaituta, irudia) VALUES
('Stranger Things', 'Zientzia fikzioa', 5, 8.7, 0, 'img/stranger-things.jpg'),
('Breaking Bad', 'Drama', 5, 9.5, 1, 'img/breaking-bad.jpg'),
('Dark', 'Zientzia fikzioa', 3, 8.7, 1, 'img/dark.jpg'),
('Wednesday', 'Fantasia', 2, 8.0, 0, 'img/wednesday.jpg'),
('The Last of Us', 'Drama', 2, 8.7, 0, 'img/the-last-of-us.jpg'),
('La casa de papel', 'Thriller', 5, 8.2, 1, 'img/la-casa-de-papel.jpg'),
('Black Mirror', 'Zientzia fikzioa', 7, 8.7, 0, 'img/black-mirror.jpg'),
('Peaky Blinders', 'Drama', 6, 8.8, 1, 'img/peaky-blinders.jpg');
```

Qué significa cada orden

Orden SQL	Explicación sencilla
`CREATE DATABASE IF NOT EXISTS`	Crea la base de datos si no existe. Si ya existe, no hace nada y no da error.
`USE`	Elige esa base de datos para trabajar dentro de ella.
`CREATE TABLE IF NOT EXISTS`	Crea la tabla si no existe.
`INT AUTO_INCREMENT PRIMARY KEY`	Número entero que sube solo (1, 2, 3...) y no se puede repetir.
`VARCHAR(100)`	Texto de hasta 100 caracteres.
`INT`	Número entero, sin decimales.
`DECIMAL(3,1)`	Número de 3 cifras con 1 decimal. Por ejemplo: 8.7.
`TINYINT(1)`	Número muy pequeño. Se usa como falso/verdadero: 0 o 1.
`NOT NULL`	Ese campo no puede quedar vacío.
`INSERT INTO ... VALUES`	Mete filas nuevas en la tabla.

Cómo ejecutar el archivo (phpMyAdmin)

1. Abre `http://localhost/phpmyadmin`.
2. Pulsa la pestaña **Importar** (arriba).
3. Pulsa **Seleccionar archivo**.
4. Elige `C:\laragon\www\Control\Control\includes\datuak.sql`.

5. Pulsa **Importar** o **Ejecutar**.
6. Espera el mensaje verde de que se ha ejecutado correctamente.

> **No hace falta crear la base de datos a mano.** El archivo ya lleva dentro `CREATE DATABASE` y `USE`, así que se crea sola al importarlo.

Comprobar que se ha creado

1. En la lista de la izquierda de phpMyAdmin, pulsa `tx_series`.
2. Pulsa la tabla `serieak`.
3. Pulsa la pestaña **Examinar**.
4. Deben aparecer **8 filas** y 7 columnas.

> **Comprobación:** si ves las 8 series (Stranger Things, Breaking Bad, Dark...), este paso está terminado.

> **Cuidado:** si ejecutas el archivo dos veces, las 8 series se insertan otra vez y aparecen duplicadas. El `INSERT` no comprueba si ya existen. Para volver a empezar: borra la base de datos `tx_series` y vuelve a importar el archivo.

6. Paso 2 — Conectar PHP con MySQL

Para que PHP pueda leer y escribir en la base de datos, primero tiene que conectarse. Eso es lo que hace el archivo `includes/konexioa.php`.

includes/konexioa.php

```php
<?php

// Datos de conexión de Laragon.
$servidor = "localhost";
$usuario = "root";
$contrasena = "";
$baseDatos = "tx_series";

// Nos conectamos a la base de datos.
$conn = mysqli_connect($servidor, $usuario, $contrasena, $baseDatos);

// Si no se puede conectar, mostramos un error y paramos.
if (!$conn) {
    die("Error de conexion: " . mysqli_connect_error());
}

?>
```

Explicación línea a línea

Código	Qué hace
`$servidor = "localhost";`	Dice que MySQL está en el mismo ordenador.
`$usuario = "root";`	Usuario con el que entramos.

`$contrasena = "";`	Contraseña vacía, la de Laragon por defecto.
`$baseDatos = "tx_series";`	Nombre de la base de datos del proyecto.
`mysqli_connect(...)`	Función de PHP que se conecta a MySQL. Devuelve la conexión o `false` si falla.
`$conn`	Guarda la conexión. Se pasa a las funciones que hacen consultas.
`if (!$conn)`	Si no hay conexión...
`die(...)`	Muestra el error y detiene la página.

Cómo probar solo la conexión

Puedes crear un archivo temporal llamado `probar_conexion.php` en la carpeta principal con este contenido:

```
<?php
require "includes/konexioa.php";
echo "Conexion correcta";
?>
```

Abre `http://localhost/Control/Control/probar_conexion.php`. Si ves «Conexion correcta», funciona. Después **borra ese archivo**, porque era solo una prueba.

> **Error de conexión:** si ves un mensaje con «Access denied» o «Unknown database», revisa el usuario, la contraseña y el nombre `tx_series`. También comprueba que MySQL está encendido en Laragon.

7. Paso 3 — Leer y guardar series (funciones)

Todas las funciones están en `includes/funtzioak.php`. Aquí están las tres que hablan con la base de datos y las que ya existían.

Función `serieakLortu` — leer todas las series

`includes/funtzioak.php`

```
function serieakLortu($conn) {

    // Hacemos la consulta: queremos todas las series.
    $resultado = mysqli_query($conn, "SELECT * FROM serieak");

    // Preparamos un array vacío.
    $series = [];

    // Vamos metiendo cada serie del resultado dentro del array.
    while ($fila = mysqli_fetch_assoc($resultado)) {
        $series[] = $fila;
    }

    return $series;
}
```

- `SELECT * FROM serieak` significa «dame todas las columnas de todas las filas de la tabla serieak».
- `mysqli_query` ejecuta la consulta y devuelve el resultado.

- `mysqli_fetch_assoc` va sacando una fila cada vez, en forma de array con los nombres de las columnas como claves.
- El bucle `while` termina cuando ya no quedan filas.
- Al final devuelve un array de series, igual que el array antiguo de `datuak.php`.

Función `serieaLortu` — buscar una serie por id

```
function serieaLortu($conn, $id) {

    $resultado = mysqli_query($conn, "SELECT * FROM serieak WHERE id = $id");

    return mysqli_fetch_assoc($resultado);
}
```

Se usa después de insertar una serie, para recuperar exactamente esa fila y mostrarla. `WHERE id = $id` significa «solo la fila cuyo id sea ese número».

Función `serieaGehitu` — guardar una serie nueva

```
function serieaGehitu($conn, $izenburua, $generoa, $denboraldiak, $balorazioa, $amaituta) {

    // En la base de datos guardamos true como 1 y false como 0.
    if ($amaituta) {
        $amaitutaBD = 1;
    } else {
        $amaitutaBD = 0;
    }

    // Hacemos la consulta para insertar la serie nueva.
    $sql = "INSERT INTO serieak (izenburua, generoa, denboraldiak, balorazioa, amaituta, irudia)
            VALUES ('$izenburua', '$generoa', '$denboraldiak', '$balorazioa', $amaitutaBD,
'img/default.jpg')";

    mysqli_query($conn, $sql);

    // Devolvemos el id de la serie que se acaba de crear.
    return mysqli_insert_id($conn);
}
```

- El formulario envía `amaituta` como 1 o 0. PHP lo convierte a `true` o `false`.
- Como en MySQL no hay verdadero/falso, se guarda **1** o **0**.
- `INSERT INTO` añade una fila nueva.
- La imagen siempre es `img/default.jpg`, igual que antes.
- `mysqli_insert_id` devuelve el `id` que MySQL ha dado a la fila nueva.

Las funciones que NO se han tocado

Función	Qué hace
`serieakErakutsi($series)`	Recorre el array y escribe las tarjetas HTML de las series.
`serieakBilatu($series, $testua)`	Recorre el array y devuelve las series cuyo título contiene el texto.
`martxanKalkulatu($series)`	Cuenta las series con `amaituta` a 0.

`batezbestekoaKalkulatu($series)`	Suma las notas y las divide entre el número de series.
`balorazioAltuena($series)`	Busca la serie con la nota más alta.

> **Truco para entenderlo:** las funciones antiguas no saben si el array viene de un archivo o de MySQL. Solo ven un array. Por eso no hay que cambiarlas.

> **Detalle:** MySQL devuelve los números como texto («8.7», «0», «1»), pero PHP los usa igual en las cuentas y en los `if`. El valor «0» se considera falso y «1» verdadero.

8. Paso 4 — Página de inicio

`index.php` es la página más sencilla. Solo tiene que pedir todas las series y mostrarlas.

index.php

```
<?php

// Cargamos las funciones.
require_once "includes/funtzioak.php";

// Cargamos la conexión a la base de datos.
require_once "includes/konexioa.php";

// Leemos todas las series de la base de datos.
$series = serieakLortu($conn);

// Cargamos la cabecera HTML.
include "header.php";

?>

<h1>Serieak</h1>

<?php

// Mostramos todas las series, incluidas las añadidas.
serieakErakutsi($series);

include "footer.php";

?>
```

Orden de ejecución

1. Se cargan las funciones
↓
2. Se carga la conexión y se crea `$conn`
↓
3. `serieakLortu($conn)` lee la tabla y devuelve el array
↓
4. Se muestra la cabecera, las series y el pie

Por qué `require_once` y `include`

Instrucción	Qué hace	Por qué se usa aquí
`require_once`	Carga un archivo una sola vez. Si no existe, da error y para.	Para funciones y conexión, que son obligatorias.
`include`	Carga un archivo. Si no existe, avisa pero sigue.	Para la cabecera y el pie, que son HTML.

> **Comprobación:** abre `http://localhost/Control/Control/index.php` y deben aparecer 8 tarjetas de series.

9. Paso 5 — Página de búsqueda

`search.php` funciona en dos momentos: primero muestra el formulario y, cuando lo envías, busca las series.

`search.php (parte principal)`

```php
<?php

require_once "includes/funtzioak.php";
require_once "includes/konexioa.php";

$series = serieakLortu($conn);

include "header.php";

?>

<h1>Serieak bilatu</h1>

<form action="search.php" method="post">
    <label for="bilaketa">Seriearen izenburua:</label>
    <input type="text" id="bilaketa" name="bilaketa" required>
    <button type="submit" name="bilatu">Bilatu</button>
</form>

<?php

if (isset($_POST["bilatu"])) {

    $testua = $_POST["bilaketa"];

    $emaitzak = serieakBilatu($series, $testua);

    if (count($emaitzak) > 0) {
        echo "<h2>Bilaketaren emaitzak</h2>";
        serieakErakutsi($emaitzak);
    } else {
        echo "<p>Ez da serierik aurkitu.</p>";
    }
}

include "footer.php";

?>
```

Qué pasa cuando buscas

1. Escribes un texto y pulsas **Bilatu**.
2. El formulario envía el texto por `POST` con el nombre `bilaketa`.
3. `isset($_POST["bilatu"])` comprueba que has pulsado el botón.
4. Se recoge el texto: `$testua = $_POST["bilaketa"];`.
5. `serieakBilatu($series, $testua)` recorre las series y compara los títulos.
6. Si hay resultados, se muestran. Si no, aparece «Ez da serierik aurkitu» (no se ha encontrado ninguna serie).

> **La búsqueda se hace en PHP, no en MySQL.** Primero se leen todas las series con `serieakLortu` y luego se filtran con la función antigua. Es la forma más sencilla y sigue usando el código que ya existía.

> **Comprobación:** busca «Dark» y debe salir una sola serie. Busca «zzz» y debe salir el mensaje de que no hay resultados.

10. Paso 6 — Página de estadísticas

`statistics.php` lee todas las series y calcula cuatro datos con las funciones de siempre.

`statistics.php (parte principal)`

```
<?php

require_once "includes/funtzioak.php";
require_once "includes/konexioa.php";

$series = serieakLortu($conn);

$serieKopurua = count($series);

// Si la tabla está vacía, evitamos errores al dividir entre 0.
if ($serieKopurua > 0) {
    $amaituGabeak = martxanKalkulatu($series);
    $batezbestekoa = batezbestekoaKalkulatu($series);
    $serieAltuena = balorazioAltuena($series);
} else {
    $amaituGabeak = 0;
    $batezbestekoa = 0;
    $serieAltuena = null;
}

include "header.php";

?>
```

Qué calcula cada cosa

Dato	Función	Cómo se calcula
Número de series	`count($series)`	Cuenta cuántos elementos tiene el array.
Series en marcha	`martxanKalkulatu`	Cuenta las que tienen `amaituta` a 0.
Nota media	`batezbestekoaKalkulatu`	Suma todas las notas y divide entre el total.
Mejor valorada	`balorazioAltuena`	Compara las notas y se queda con la más alta.

El `if ($serieKopurua > 0)`

Es una protección. Si la tabla estuviera vacía, la media intentaría dividir entre cero y daría error. Con este `if`, si no hay series se muestran 0 y «Ez dago serierik» en lugar de romperse.

> **Comprobación con los datos iniciales:**
>
> - Serie kopurua: **8**
> - Martxan dauden serieak: **4**
> - Batez besteko balorazioa: **8.66**
> - Baloraziorik altuena: **Breaking Bad - 9.5**

11. Paso 7 — Página de añadir serie

Es la página más completa: muestra un formulario, recoge lo que escribes y lo guarda en MySQL.

Los campos del formulario

Etiqueta en la web	`name` del campo	Tipo
Izenburua (título)	`izenburua`	Texto
Generoa (género)	`generoa`	Desplegable
Denboraldi kopurua (temporadas)	`denboraldiak`	Número
Balorazioa (nota)	`balorazioa`	Número con decimales
Egoera (estado)	`amaituta`	0 = en marcha, 1 = terminada

> El atributo `name` es muy importante: es el nombre con el que PHP recibe el dato en `$_POST`. Si cambias el `name`, tienes que cambiar también el PHP.

Qué pasa al pulsar «Seriea gehitu»

`add_series.php (parte de PHP)`

```php
<?php

if (isset($_POST["gehitu"])) {

    // Recogemos los datos del formulario.
    $izenburua = $_POST["izenburua"];
    $generoa = $_POST["generoa"];
    $denboraldiak = $_POST["denboraldiak"];
    $balorazioa = $_POST["balorazioa"];

    // Convertimos el valor recibido en booleano.
    if ($_POST["amaituta"] == 1) {
        $amaituta = true;
    } else {
        $amaituta = false;
    }

    // Se guarda en la base de datos y nos devuelve su id.
    $idBerria = serieaGehitu(
        $conn,
        $izenburua,
        $generoa,
        $denboraldiak,
        $balorazioa,
        $amaituta
    );

    echo "<h2>Serie berria</h2>";

    // Se recupera de la base de datos usando su id.
    $serieBerria = serieaLortu($conn, $idBerria);

    // Mostramos la nueva serie.
    serieakErakutsi([$serieBerria]);
}

include "footer.php";

?>
```

Explicación paso a paso

1. `isset($_POST["gehitu"])` comprueba que has enviado el formulario.
2. Se recogen los cuatro datos de texto y número.
3. El desplegable de estado envía 0 o 1. Se convierte a `false` o `true`.
4. `serieaGehitu` hace el `INSERT` y devuelve el id nuevo.
5. `serieaLortu` busca esa fila por su id.
6. `serieakErakutsi([$serieBerria])` la muestra. Los corchetes crean un array con una sola serie, porque la función espera un array.

> **Comprobación:** añade una serie de prueba. Debe aparecer en la misma página y, al ir a `index.php`, seguir ahí. Eso demuestra que se ha guardado en la base de datos.

12. Recorrido completo de los datos

Este es el camino que siguen los datos desde que añades una serie hasta que la ves en las listas:

Cuando añades una serie

Escribes en el formulario de `add_series.php`

↓

El navegador envía los datos por `POST`

↓

PHP los recoge en `$_POST`

↓

`serieaGehitu($conn, ...)` hace un `INSERT`

↓

MySQL guarda la fila en la tabla `serieak`

↓

`serieaLortu($conn, $id)` la lee y se muestra

Cuando abres index, search o statistics

La página carga `konexioa.php` y crea `$conn`

↓

`serieakLortu($conn)` hace un `SELECT * FROM serieak`

↓

MySQL devuelve todas las filas

↓

PHP las convierte en un array de series

↓

Las funciones de mostrar, buscar o calcular trabajan con ese array

↓

El navegador recibe el HTML final

> **Idea clave:** la base de datos es la única fuente de verdad. Ya no se usa la sesión para guardar series. Todo lo que ves en pantalla viene de MySQL.

Qué archivo hace cada cosa

Archivo	Responsabilidad
`konexioa.php`	Abrir la puerta a MySQL.
`funtzioak.php`	Hablar con MySQL y hacer los cálculos.
`index.php`	Mostrar la lista completa.
`search.php`	Filtrar la lista por título.
`statistics.php`	Calcular y mostrar los totales.
`add_series.php`	Recoger el formulario y guardar.

13. Pruebas de funcionamiento

Haz estas pruebas en orden. Si todas salen bien, el ejercicio está terminado.

Nº	Qué haces	Qué debe pasar
1	Abrir `index.php`.	Se ven 8 series con su imagen, género, temporadas, nota y estado.
2	Abrir `statistics.php`.	8 series, 4 en marcha, media 8.66, mejor valorada Breaking Bad (9.5).
3	En `search.php`, buscar «Dark».	Aparece solo la serie Dark.
4	Buscar «zzz».	Aparece «Ez da serierik aurkitu».
5	En `add_series.php`, añadir una serie de prueba.	Aparece el mensaje «Serie berria» y la tarjeta de la serie nueva.
6	Volver a `index.php`.	La serie de prueba sigue apareciendo.
7	Abrir phpMyAdmin y examinar `serieak`.	La serie de prueba está como una fila más.
8	Borrar la serie de prueba.	La tabla vuelve a tener 8 filas.

Cómo borrar la serie de prueba

En phpMyAdmin:

1. Entra en `tx_series` → `serieak` → **Examinar**.
2. Busca la fila de tu serie de prueba.
3. Pulsa **Borrar** y confirma.

O con una consulta SQL:

```
USE tx_series;
DELETE FROM serieak WHERE izenburua = 'Mi serie de prueba';
```

> **Importante:** si añades una serie y no la borras, las estadísticas cambiarán. Para que las pruebas coincidan con la guía, borra la serie de prueba al terminar.

14. Problemas frecuentes y solución

Qué ves	Causa probable	Solución
«Error de conexion: Access denied»	Usuario o contraseña incorrectos.	Revisa `$usuario` y `$contrasena` en `konexioa.php`.
«Unknown database 'tx_series'»	La base de datos no existe.	Importa `includes/datuak.sql` en phpMyAdmin.
«Table 'tx_series.serieak' doesn't exist»	La base existe pero la tabla no.	Vuelve a importar el archivo SQL completo.
Página en blanco	Hay un error de PHP y no se muestra.	Revisa que Laragon esté encendido y mira los errores en los logs de Laragon.
«Call to undefined function serieakLortu»	No se ha cargado `funtzioak.php`.	Comprueba el `require_once "includes/funtzioak.php";`.
«Undefined variable $conn»	No se ha cargado `konexioa.php`.	Comprueba el `require_once "includes/konexioa.php";`.
Las series salen duplicadas	Se ha ejecutado el SQL más de una vez.	Borra las filas duplicadas o borra la base y vuelve a importar.
No sale la imagen de una serie	La ruta de `irudia` no existe.	Comprueba el nombre del archivo dentro de `img/`.
La media da error o no sale	La tabla está vacía.	La página ya lo controla. Si no hay series, muestra 0.
Los cambios no aparecen	Estás mirando otra carpeta o el navegador tiene caché.	Recarga con Ctrl + F5 y revisa la URL.
Funciona phpMyAdmin pero no la web	Datos de conexión distintos en `konexioa.php`.	Revisa servidor, usuario, contraseña y nombre de la base.

> **Consejo:** cuando algo falle, no cambies cinco cosas a la vez. Cambia una, recarga y mira si se ha arreglado. Así sabes qué era el problema.

15. Diccionario de nombres en euskera

El proyecto está en euskera. Los comentarios están en español, pero los nombres de funciones, variables y columnas se han mantenido en euskera para no romper el código.

Columnas de la base de datos

Nombre	Significado
`id`	Identificador único.
`izenburua`	Título.
`generoa`	Género.

`denboraldiak`	Temporadas.
`balorazioa`	Valoración / nota.
`amaituta`	Terminada.
`irudia`	Imagen.

Funciones y variables

Nombre	Significado
`serieak`	Las series.
`seriea`	La serie.
`serieakLortu`	Obtener las series.
`serieaLortu`	Obtener una serie.
`serieaGehitu`	Añadir una serie.
`serieakErakutsi`	Mostrar las series.
`serieakBilatu`	Buscar series.
`martxanKalkulatu`	Calcular las que están en marcha.
`batezbestekoaKalkulatu`	Calcular la media.
`balorazioAltuena`	La valoración más alta.
`serieKopurua`	Cantidad de series.
`testua`	Texto.
`emaitzak`	Resultados.
`bilaketa`	Búsqueda.
`berria`	Nueva.
`kopurua`	Cantidad.

Textos del menú

En la web	Significado
Hasiera	Inicio.
Bilatu	Buscar.
Seriea gehitu	Añadir serie.
Estatistikak	Estadísticas.
Serie berria	Serie nueva.
Ez da serierik aurkitu	No se ha encontrado ninguna serie.

16. Checklist final y cómo explicar el ejercicio

Checklist final

- ☐ Laragon está encendido y MySQL funciona.
- ☐ La base de datos `tx_series` existe.
- ☐ La tabla `serieak` tiene 7 columnas y 8 filas iniciales.
- ☐ `konexioa.php` conecta sin errores.
- ☐ `index.php` muestra las 8 series.
- ☐ `search.php` encuentra «Dark» y avisa cuando no hay resultados.
- ☐ `statistics.php` muestra 8, 4, 8.66 y Breaking Bad 9.5.
- ☐ `add_series.php` guarda una serie nueva.
- ☐ La serie nueva sigue apareciendo después de recargar y en phpMyAdmin.
- ☐ La serie de prueba se ha borrado.
- ☐ Ninguna página carga ya `includes/datuak.php`.

Cómo explicar el ejercicio en 1 minuto

1. **Qué hace:** una web para ver, buscar, añadir y calcular estadísticas de series.
2. **Antes:** los datos estaban en un array dentro de `datuak.php` y se guardaban en la sesión.
3. **Ahora:** los datos están en MySQL, en la base `tx_series` y la tabla `serieak`.
4. **Cómo se conecta:** `konexioa.php` crea `$conn` con `mysqli_connect`.
5. **Cómo funciona:** `serieakLortu` lee con `SELECT`, `serieaGehitu` guarda con `INSERT` y las funciones antiguas siguen mostrando y calculando.

Qué has aprendido

Concepto	Dónde lo has usado
Crear una base de datos y una tabla	`datuak.sql`
Conectar PHP con MySQL	`konexioa.php`
Leer datos con `SELECT`	`serieakLortu` y `serieaLortu`
Guardar datos con `INSERT`	`serieaGehitu`
Recorrer resultados con `while`	`mysqli_fetch_assoc`
Pasar datos de una página a otra	Formulario `POST` de `add_series.php`
Separar responsabilidades	Conexión, funciones y páginas en archivos distintos

> **Enhorabuena.** Si has llegado hasta aquí con las pruebas en verde, has pasado un proyecto que usaba un archivo de datos a uno que usa una base de datos real.

Anexo — Comandos SQL útiles

Estos comandos se pueden escribir en la pestaña **SQL** de phpMyAdmin. Sirven para consultar o limpiar la base de datos mientras practicas.

Ver todas las series

```
USE tx_series;
SELECT * FROM serieak;
```

Muestra todos los campos y todas las filas de la tabla.

Ver solo algunos campos

```
SELECT id, izenburua, balorazioa
FROM serieak
ORDER BY balorazioa DESC;
```

Muestra el id, el título y la nota, ordenados de mayor a menor nota. `DESC` significa descendente.

Buscar una serie concreta

```
SELECT * FROM serieak WHERE izenburua = 'Dark';
```

Muestra solo la serie cuyo título es exactamente «Dark».

Borrar una serie por su id

```
DELETE FROM serieak WHERE id = 9;
```

Borra la fila con id 9. Cambia el número por el que necesites. **Cuidado:** el `DELETE` borra de verdad; no se puede deshacer.

Contar cuántas series hay

```
SELECT COUNT(*) AS total FROM serieak;
```

Devuelve el número de filas de la tabla.

Vaciar la tabla sin borrarla

```
TRUNCATE TABLE serieak;
```

Deja la tabla vacía y reinicia el contador de `id`. Después tendrías que volver a insertar los datos iniciales.

Borrar toda la base de datos

```
DROP DATABASE tx_series;
```

Borra la base de datos entera, con su tabla y sus datos. Solo se usa si quieres empezar de cero. Después hay que volver a importar `datuak.sql`.

> **Antes de usar `DELETE`, `TRUNCATE` o `DROP`**, asegúrate de que estás en la base de datos `tx_series` y de que no necesitas los datos. Son órdenes que borran información.

> **Resumen:** para trabajar cada día solo necesitas Laragon encendido y abrir `http://localhost/Control/Control/index.php`. La base de datos ya está creada. El archivo `datuak.sql` solo se usa para instalarla o restaurarla.